

Basic Concepts:

1. How is code reuse achieved through inheritance?

In Java, inheritance allows a class (subclass/child) to acquire properties and behaviors (fields and methods) from another class (superclass/parent) using the `extends` keyword. This enables code reuse, avoiding duplication and promoting modular design.

2. What are the types of inheritance in Java? Why does Java not support multiple inheritance?

1. Single Inheritance

One class inherits from one superclass.

```
class Animal {  
    void eat() {}  
}  
class Dog extends Animal {  
    void bark() {}  
}
```

2. Multilevel Inheritance

A class inherits from a class which is itself a subclass.

```
class Animal {  
    void eat() {}  
}  
class Dog extends Animal {  
    void bark() {}  
}
```

```
}  
class Puppy extends Dog {  
    void weep() {}  
}
```

3. Hierarchical Inheritance

Multiple classes inherit from a single superclass.

```
class Animal {  
    void eat() {}  
}  
class Dog extends Animal {}  
class Cat extends Animal {}
```

4. Hybrid Inheritance (Through Interfaces)

Combination of multiple types using **interfaces** (since Java doesn't support multiple class inheritance).

```
interface A { void methodA(); }  
interface B { void methodB(); }  
class C implements A, B {  
    public void methodA() {}  
    public void methodB() {}  
}
```

3. Define a class Person with attributes name and age.

```
public class Main {  
    static class Person {  
        String name;  
        int age;
```

```
public Person(String name, int age) {  
    this.name = name;  
    this.age = age;  
}  
  
public void displayInfo() {  
    System.out.println("Name: " + name);  
    System.out.println("Age: " + age);  
}  
}  
  
public static void main(String[] args) {  
    Person p = new Person("Yajju", 24);  
    p.displayInfo();  
}  
}
```

Output[Clear](#)

Name: Yajju
Age: 24

=== Code Execution Successful ===

4. Create a subclass Employee that inherits from Person and adds an attribute employeeId.

```
// Question number 4

public class MainEmployee {

    // Superclass Person

    static class Person {

        String name;

        int age;

        public Person(String name, int age) {

            this.name = name;

            this.age = age;

        }

        public void displayInfo() {

            System.out.println("Name: " + name);

            System.out.println("Age: " + age);

        }

    }

}

// Subclass Employee that inherits from Person
```

```
static class Employee extends Person {

    String employeeId;

    public Employee(String name, int age, String employeeId) {

        super(name, age); // Call superclass constructor

        this.employeeId = employeeId;

    }

    // Override displayInfo to include employeeId

    @Override

    public void displayInfo() {

        super.displayInfo(); // Call Person's displayInfo

        System.out.println("Employee ID: " + employeeId);

    }

}

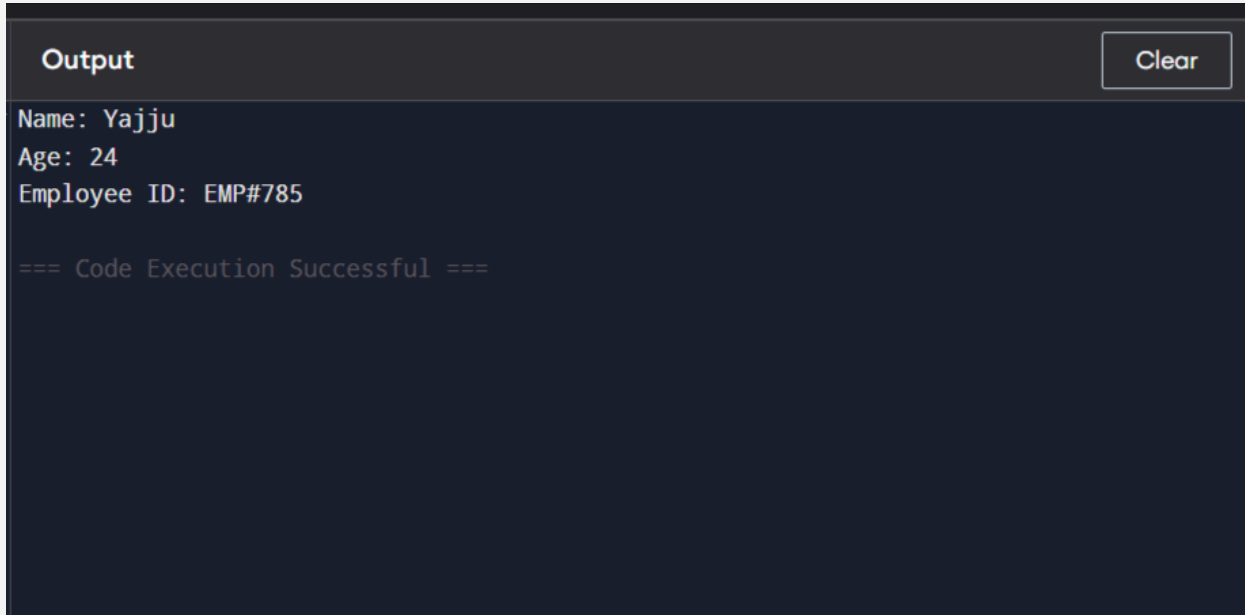
public static void main(String[] args) {

    Employee e = new Employee("Yajju", 24, "EMP#785");

    e.displayInfo();

}

}
```



The screenshot shows a dark-themed IDE output window. At the top, there is a header bar with the word "Output" on the left and a "Clear" button on the right. Below the header, the output text is displayed in a monospaced font. It shows the program's output: "Name: Yajju", "Age: 24", and "Employee ID: EMP#785". After a blank line, it shows "=== Code Execution Successful ===".

```
Output Clear  
Name: Yajju  
Age: 24  
Employee ID: EMP#785  
  
=== Code Execution Successful ===
```

Method Overriding:

5. Define a class Animal with a method makeSound().

```
public class MainAnimal {  
  
    // Class Animal  
  
    static class Animal {  
  
        public void makeSound() {  
  
            System.out.println("The animal makes a sound.");  
  
        }  
  
    }  
  
  
    public static void main(String[] args) {  
  
        Animal a = new Animal();  
  
    }  
}
```

```
a.makeSound();  
  
}  
  
}
```

Output[Clear](#)

The animal makes a sound.

=== Code Execution Successful ===

6. Create subclasses Dog and Cat that inherit from Animal and override the makeSound() method with barking and meowing, respectively.

```
public class MainDogCat {  
  
    // Superclass Animal  
  
    static class Animal {  
  
        public void makeSound() {  
  
            System.out.println("The animal makes a sound.");  
  
        }  
  
    }  
  
}
```

```
    }

}

// Subclass Dog

static class Dog extends Animal {

    @Override

    public void makeSound() {

        System.out.println("The dog barks.");

    }

}

// Subclass Cat

static class Cat extends Animal {

    @Override

    public void makeSound() {

        System.out.println("The cat meows.");

    }

}

public static void main(String[] args) {

    Animal a1 = new Dog();

    Animal a2 = new Cat();

}
```



```
a1.makeSound(); // Output: The dog barks.  
  
a2.makeSound(); // Output: The cat meows.  
  
}  
  
}
```

Output Clear

```
The dog barks.  
The cat meows.  
  
=== Code Execution Successful ===
```

Multilevel Inheritance:

7. Create a class A with a method display() that prints "Class A".

```
public class Mainclass_A {  
  
    // Base class A  
  
    static class A {  
  
        public void display() {
```

```
        System.out.println("Class A");
    }

}

public static void main(String[] args) {

    A obj = new A();

    obj.display(); // Output: Class A

}

}
```

Output

Class A

=== Code Execution Successful ===

8. Create a subclass B that inherits from A and overrides display() to print "Class B".

```
public class Mainsubclass_B {
    // Base class A
    static class A {
        public void display() {
```

```
        System.out.println("Class A");
    }
}

// Subclass B inheriting from A
static class B extends A {
    @Override
    public void display() {
        System.out.println("Class B");
    }
}

public static void main(String[] args) {
    B obj = new B();
    obj.display(); // Output: Class B
}
}
```

Output Clear

Class B

=== Code Execution Successful ===

9. Create another subclass C that inherits from B and overrides display() to print "Class C".

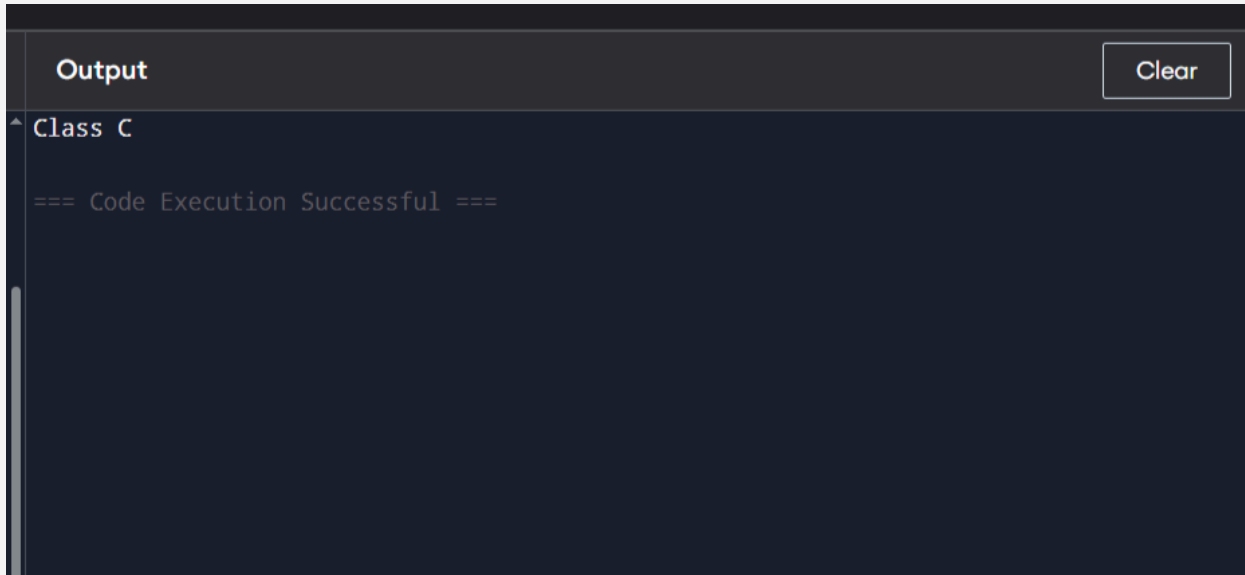
```
public class Mainsubclass_C {
```

```
// Base class A
static class A {
    public void display() {
        System.out.println("Class A");
    }
}

// Subclass B inherits from A
static class B extends A {
    @Override
    public void display() {
        System.out.println("Class B");
    }
}

// Subclass C inherits from B
static class C extends B {
    @Override
    public void display() {
        System.out.println("Class C");
    }
}

public static void main(String[] args) {
    C obj = new C();
    obj.display(); // Output: Class C
}
}
```



The screenshot shows a dark-themed IDE output window. At the top, there is a tab labeled 'Output' and a 'Clear' button. Below the tab, the text 'Class C' is displayed with a small upward-pointing arrow to its left. Further down, the text '=== Code Execution Successful ===' is shown in a monospaced font.

10. Create an object of class C and call its display() method.

```
public class Mainclass_C {  
  
    // Base class A  
  
    static class A {  
  
        public void display() {  
  
            System.out.println("Class A");  
  
        }  
  
    }  
  
    // Subclass B inherits from A  
  
    static class B extends A {  
  
        @Override  
  
        public void display() {  
  
            System.out.println("Class B");  
  
        }  
  
    }  
  
}
```

```
}

// Subclass C inherits from B

static class C extends B {

    @Override

    public void display() {

        System.out.println("Class C");

    }

}

public static void main(String[] args) {

    // Creating object of class C

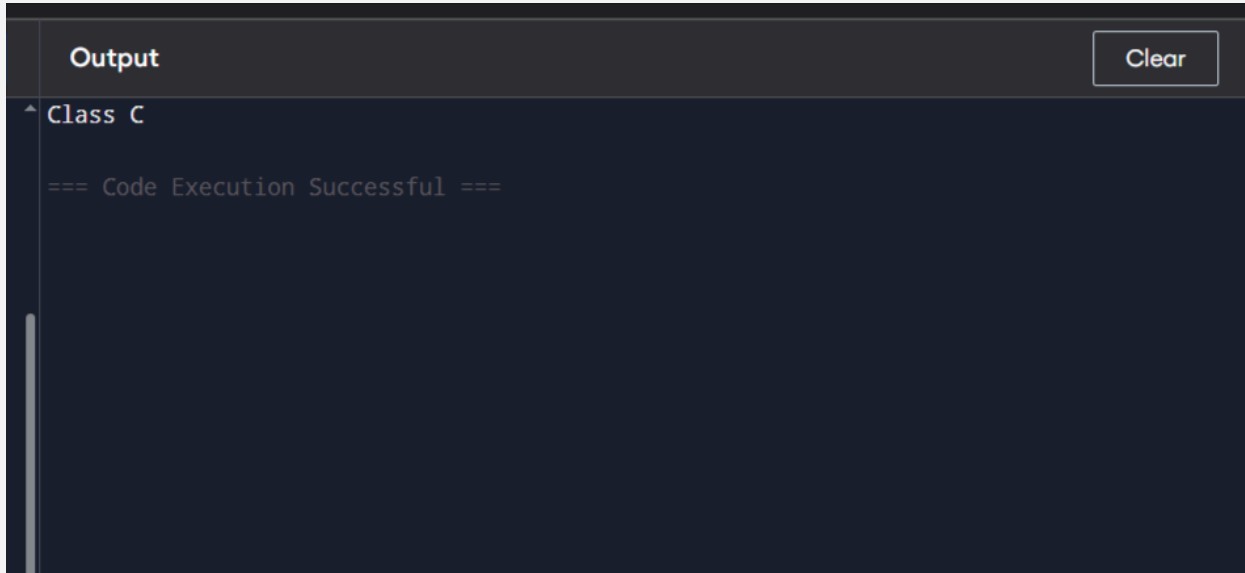
    C obj = new C();

    // Calling display() method

    obj.display(); // Output: Class C

}

}
```

A screenshot of an IDE's Output window. The window has a dark background. At the top, there is a tab labeled 'Output' and a 'Clear' button. Below the tab, the text 'Class C' is visible, followed by '=== Code Execution Successful ==='.

```
Output
Clear
^ Class C
=== Code Execution Successful ===
```

super() keyword:

11. Implement a class Shape with attributes color and method area().

```
//Question number 11

public class MainShape {

    // Superclass Shape

    static class Shape {

        String color;

        // Constructor

        public Shape(String color) {
```

```
        this.color = color;

    }

    // Method to return area (default implementation)

    public void area() {

        System.out.println("Area of shape is undefined.");

    }

}

public static void main(String[] args) {

    Shape s = new Shape("Red");

    s.area(); // Output: Area of shape is undefined.

}

}
```

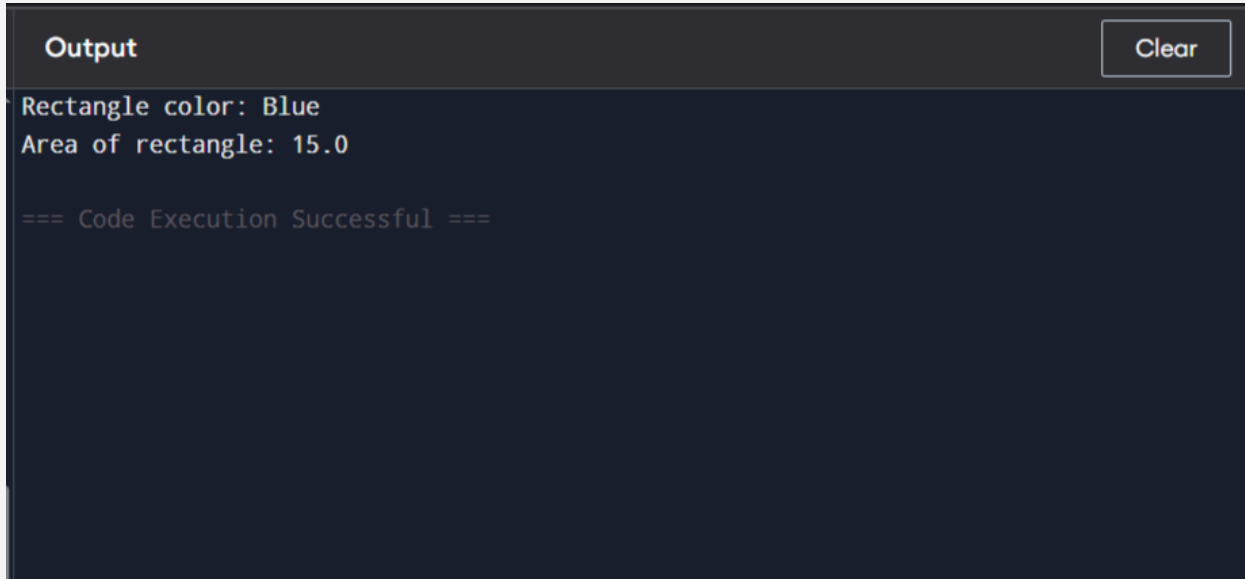

Output Clear

```
Area of shape is undefined.  
  
=== Code Execution Successful ===
```

12. Create a subclass Rectangle that inherits from Shape and adds attributes length and width.

```
public class MainRectangle {  
    // Superclass Shape  
    static class Shape {  
        String color;  
  
        // Constructor  
        public Shape(String color) {  
            this.color = color;  
        }  
  
        // Method to print area info (general placeholder)  
        public void area() {  
            System.out.println("Area of shape is undefined.");  
        }  
    }  
  
    // Subclass Rectangle inherits from Shape  
    static class Rectangle extends Shape {
```

```
double length;  
double width;  
  
// Constructor  
public Rectangle(String color, double length, double width) {  
    super(color); // Call Shape's constructor  
    this.length = length;  
    this.width = width;  
}  
  
// Override area method  
@Override  
public void area() {  
    double result = length * width;  
    System.out.println("Rectangle color: " + color);  
    System.out.println("Area of rectangle: " + result);  
}  
  
public static void main(String[] args) {  
    Rectangle r = new Rectangle("Blue", 5.0, 3.0);  
    r.area(); // Output: Rectangle color: Blue, Area: 15.0  
}  
}
```



```
Output
Rectangle color: Blue
Area of rectangle: 15.0

=== Code Execution Successful ===
```

13. Use `super()` to initialize the color attribute in the Rectangle constructor.

```
public class Mainsuper {

    // Superclass Shape

    static class Shape {

        String color;

        // Constructor

        public Shape(String color) {

            this.color = color;

        }

        public void area() {

            System.out.println("Area of shape is undefined.");

        }

    }

}
```

```
    }

}

// Subclass Rectangle inherits from Shape

static class Rectangle extends Shape {

    double length;

    double width;

    // Constructor using super() to initialize color

    public Rectangle(String color, double length, double width) {

        super(color); // Initializes 'color' from Shape

        this.length = length;

        this.width = width;

    }

    @Override

    public void area() {

        double result = length * width;

        System.out.println("Rectangle color: " + color);

        System.out.println("Area of rectangle: " + result);

    }

}
```

```
public static void main(String[] args) {  
  
    Rectangle r = new Rectangle("Green", 4.5, 2.0);  
  
    r.area();  
  
}  
}
```

Output Clear

Rectangle color: Green
Area of rectangle: 9.0

=== Code Execution Successful ===

Final keyword in Java:

14. Define a class, “Animal” as a final class. Try creating a subclass “Dog” that inherits from class “Animal” and observe the result.

```
public class MainAnimalDog {
```

```
// Final class - cannot be extended

public final static class Animal {

    public void sound() {

        System.out.println("Animal makes a sound.");

    }

}

// Attempt to inherit from final class (this will cause a compile-time
error)

/*

static class Dog extends Animal {

    public void sound() {

        System.out.println("Dog barks.");

    }

}

*/

public static void main(String[] args) {

    Animal a = new Animal();

    a.sound();

}

}
```

Output Clear

Animal makes a sound.

=== Code Execution Successful ===

15. Define a class `Vehicle` as `final` with a method `start()`. Attempt to create a subclass `Car` that inherits from `Vehicle` and observe the result.

```
public class MainVehicle {  
  
    // Final class Vehicle  
  
    public final static class Vehicle {  
  
        public void start() {  
  
            System.out.println("Vehicle is starting...");  
  
        }  
  
    }  
  
}
```



```
/*
```

```
public static class Car extends Vehicle {  
  
    public void start() {  
  
        System.out.println("Car is starting...");  
  
    }  
  
}  
  
*/  
  
public static void main(String[] args) {  
  
    Vehicle v = new Vehicle();  
  
    v.start();  
  
}  
}
```



```
Output Clear  
Vehicle is starting...  
=== Code Execution Successful ===
```

Method Overloading:

16. Create a class AreaCalculator with method calculateArea and perform the following tasks:
 - a. Create an overloaded method named calculateArea to compute and display the area of a rectangle using length and width.
 - b. Overload the calculateArea method to calculate and display the area of a square using the side length.
 - c. Overload the calculateArea method again to calculate and display the area of a triangle using base and height.

```
public class MainAreaCalculator {  
  
    // AreaCalculator class with overloaded calculateArea methods
```

```
static class AreaCalculator {

    // Rectangle: length × width

    public void calculateArea(double length, double width) {

        double area = length * width;

        System.out.println("Area of rectangle: " + area);

    }

    // Square: side × side

    public void calculateArea(double side) {

        double area = side * side;

        System.out.println("Area of square: " + area);

    }

    // Triangle: 0.5 × base × height

    public void calculateArea(double base, double height, boolean
isTriangle) {

        double area = 0.5 * base * height;

        System.out.println("Area of triangle: " + area);

    }

}

public static void main(String[] args) {

    AreaCalculator calculator = new AreaCalculator();
```

```
calculator.calculateArea(5.0, 3.0);           // Rectangle  
  
calculator.calculateArea(4.0);               // Square  
  
calculator.calculateArea(6.0, 2.5, true);    // Triangle  
  
}  
  
}
```

Output[Clear](#)

```
Area of rectangle: 15.0  
Area of square: 16.0  
Area of triangle: 7.5
```

```
=== Code Execution Successful ===
```